

1. TypeScript

TypeScript is a **superset of JavaScript** that adds **static typing**. It allows developers to define types for variables, functions, and objects, which helps in detecting errors during development instead of runtime.

Example:

None

```
let name: string = "Sai";  
name = 10; // ❌ Error
```

Here, TypeScript restricts name to only string values, preventing incorrect assignments.

2. Need for TypeScript

JavaScript is **dynamically typed**, meaning variables can change types, which may cause runtime errors. TypeScript solves this by enforcing types.

Example:

None

```
let x = "hello";  
x = 10; // allowed in JavaScript
```

None

```
let x: string = "hello";  
x = 10; // ❌ Error in TypeScript
```

TypeScript ensures type consistency, reducing bugs.

3. Type Inference

TypeScript automatically detects the type of a variable based on its value.

Example:

None

```
let message = "Hello";
```

TypeScript infers message as string even without explicitly defining it.

4. Explicit Typing

Developers can manually define types when needed.

Example:

```
None
let age: number = 20;
let isStudent: boolean = true;
```

This improves readability and prevents accidental misuse.

5. Interfaces

Interfaces define the **structure (shape)** of an object.

Example:

```
None
interface User {
  name: string;
  id: number;
}

const user: User = {
  name: "Sai",
  id: 1,
};
```

The object must match the interface structure, otherwise TypeScript throws an error.

6. Classes

Classes in TypeScript can use types and follow object-oriented programming.

Example:

```
None
class Person {
  name: string;
```

```
constructor(name: string) {  
  this.name = name;  
}  
}
```

TypeScript ensures class properties follow defined types.

7. Functions with Types

Functions can have typed parameters and return values.

Example:

```
None  
function add(a: number, b: number): number {  
  return a + b;  
}
```

Ensures correct input and output types.

8. Primitive Types

TypeScript supports basic types from JavaScript.

Example:

```
None  
let str: string = "Hello";  
let num: number = 10;  
let flag: boolean = true;
```

Also includes null, undefined, and special types like any, unknown, void, never.

9. Union Types

Union types allow a variable to have multiple possible types.

Example:

None

```
function printId(id: number | string) {  
  console.log(id);  
}
```

id can be either a number or string.

10. Type Checking (typeof)

Used to check variable type at runtime.

Example:

None

```
function check(x: string | number) {  
  if (typeof x === "string") {  
    console.log("string");  
  } else {  
    console.log("number");  
  }  
}
```

✓ Explanation:

Helps handle multiple types safely.

11. Generics

Generics allow creating reusable and type-safe components.

Example:

None

```
interface Box<T> {  
  value: T;  
}
```

```
let box1: Box<string> = { value: "Hello" };  
let box2: Box<number> = { value: 10 };
```

T acts as a placeholder for any type.

12. Structural Typing

TypeScript checks **object structure (shape)**, not the name.

Example:

```
None
interface Point {
  x: number;
  y: number;
}

function printPoint(p: Point) {
  console.log(p.x, p.y);
}

printPoint({ x: 10, y: 20 }); //  works
```

Even if object is not explicitly declared as Point, it works if structure matches.

13. Special Types

Example:

```
None
let data: any = 10;
data = "hello";

let value: unknown;

function noReturn(): void {}

function error(): never {
  throw new Error("error");
}
```

- any → no restrictions
- unknown → safer than any
- void → no return
- never → never occurs